MIDTERM ESSAY · 503073 WEB PROGRAMMING & APPLICATIONS

Containers in Web Development and Deployment (Docker)

Trần Trung Hiếu · 52400047
Nguyễn Đức Nhật Huy · 52400122
Instructor: MSc. Nguyễn Thái Duy
Faculty of Information Technology, Ton Duc Thang University



WHY THIS TOPIC

“It works on my machine”

The code is fine. The environment around it drifts. Containers package the application with everything it needs, so it behaves the same on any host.

71.1%

of respondents use Docker (Stack Overflow Developer Survey, 2025)

82%

of container users run Kubernetes in production (CNCF survey, 2025)

OUTLINE

We built it, broke it on purpose and measured it

Theory

How containers work, from Linux kernel features to orchestration.

Comparison

Virtual machines, other engines, Compose and Kubernetes.

DockMart

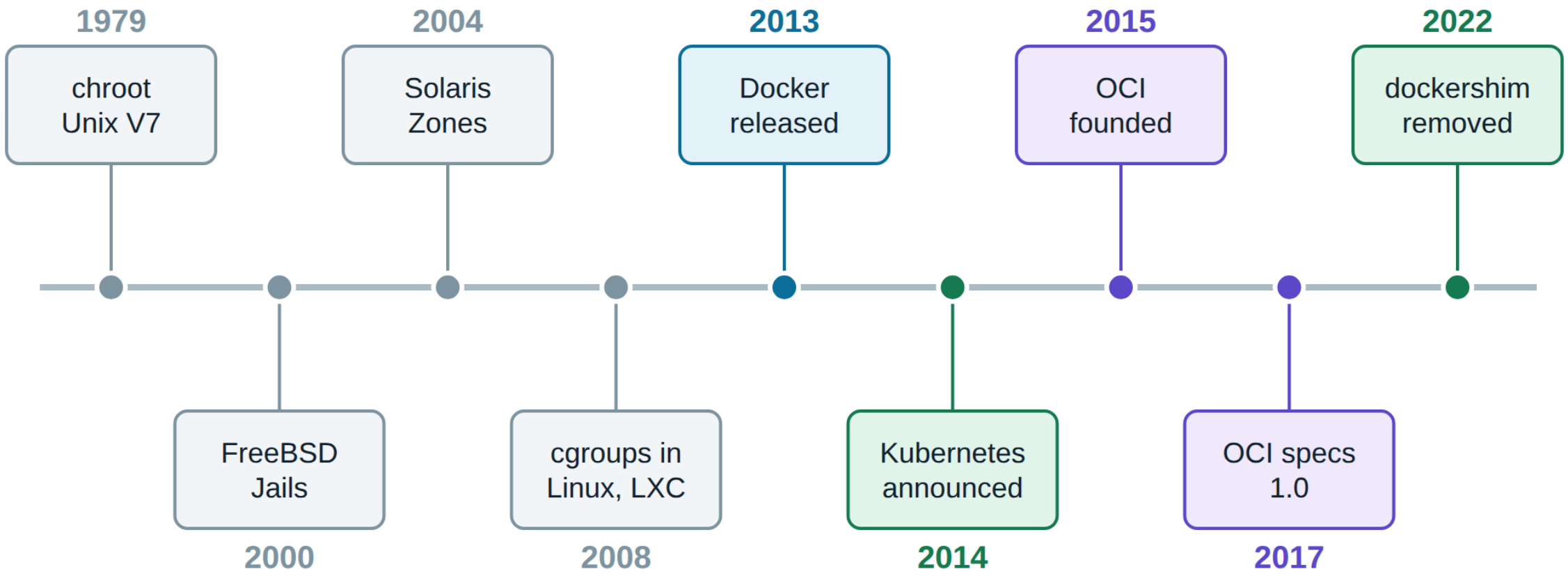
A multi-container online store built to test each claim.

Results

Image size, scaling, failures, rolling updates, autoscaling.

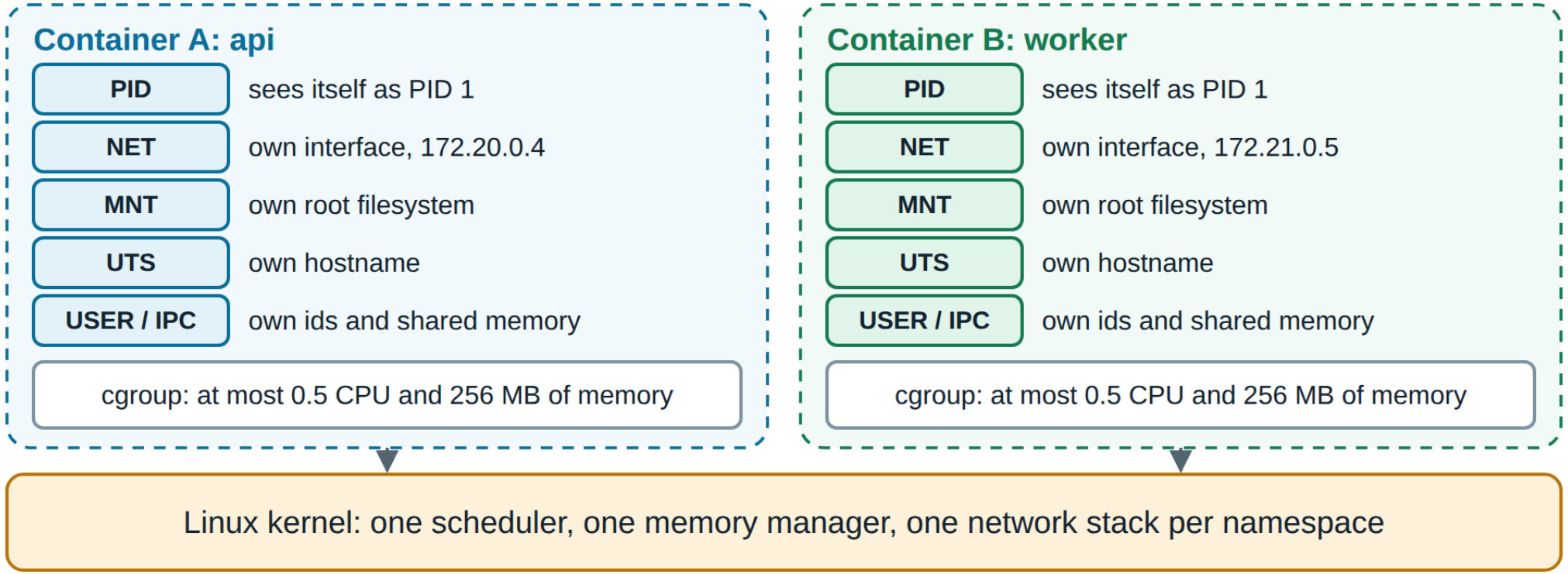
HISTORY

Docker made containers easy, it did not invent them



THEORY

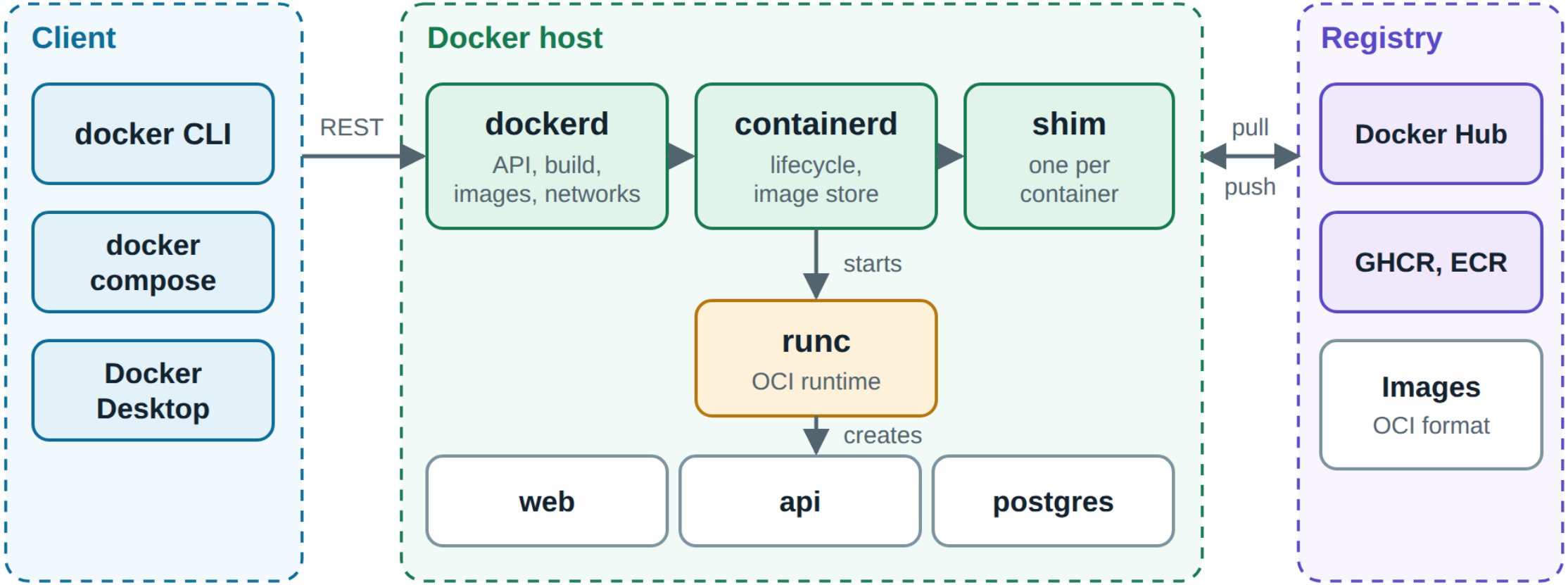
A container is a Linux process, not a small VM



Namespaces limit what it can see. **Cgroups** limit what it can use.

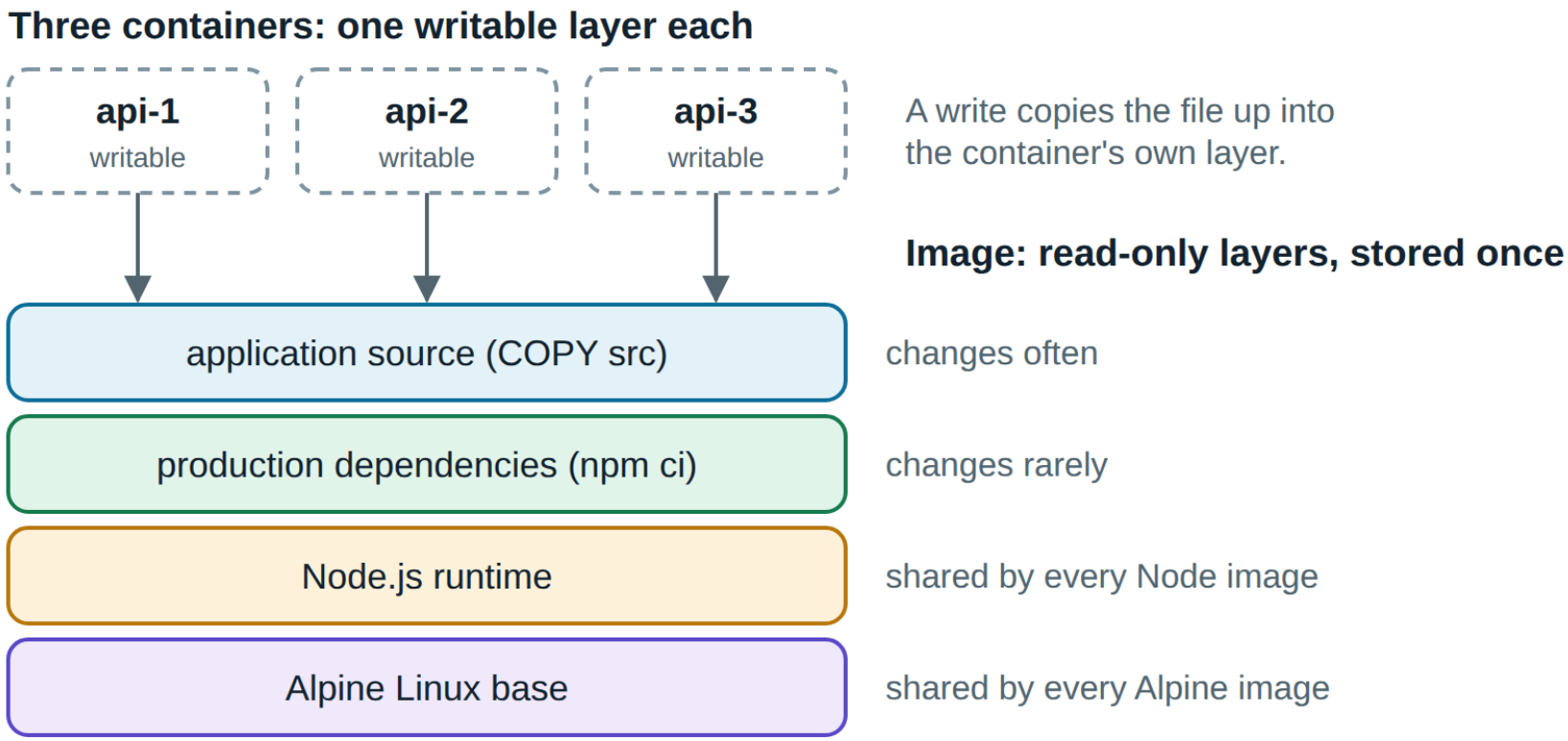
THEORY

Docker is a client talking to a stack of runtimes



THEORY

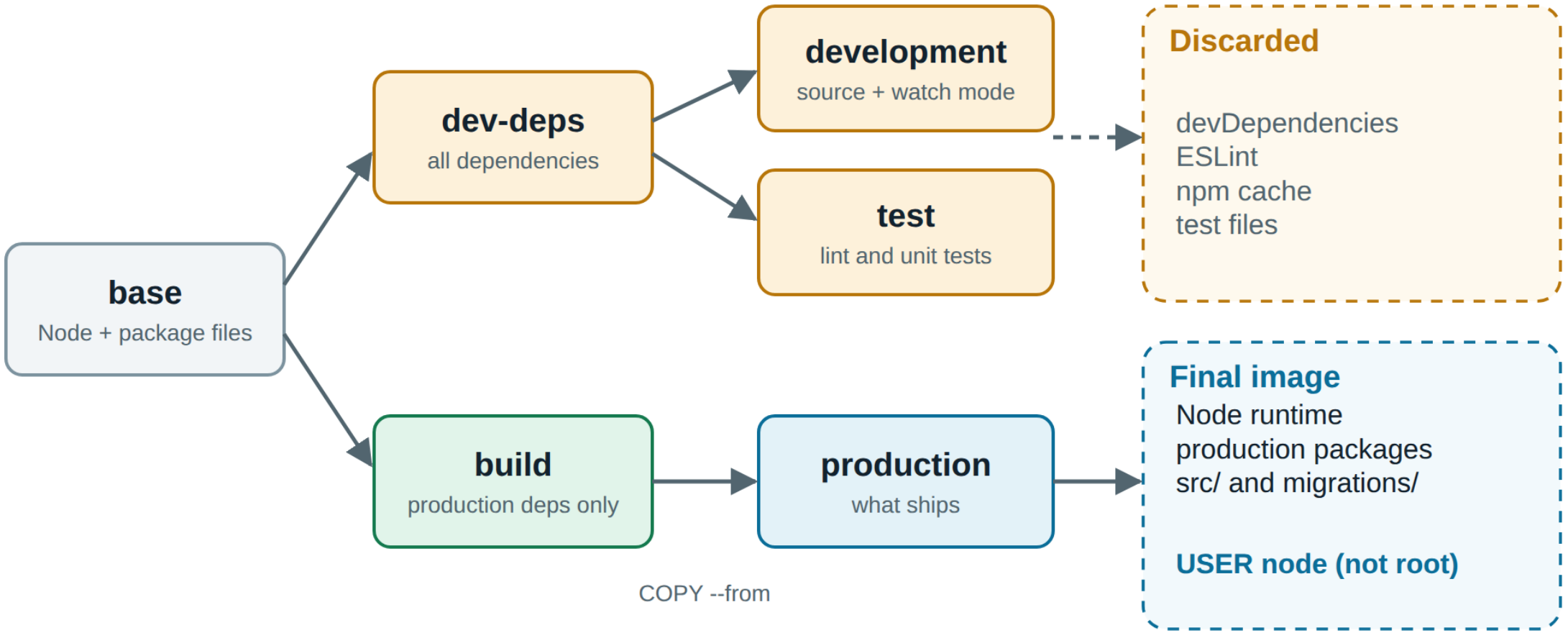
Images are shared layers



Three replicas share one copy on disk, and a change rebuilds only that layer and those after it.

THEORY

Multi-stage builds keep tools out of production



THEORY

Two run-time details that catch teams out

Signals

docker stop sends SIGTERM, then SIGKILL after 10 seconds. Starting through npm can swallow the signal, so open requests get cut off.

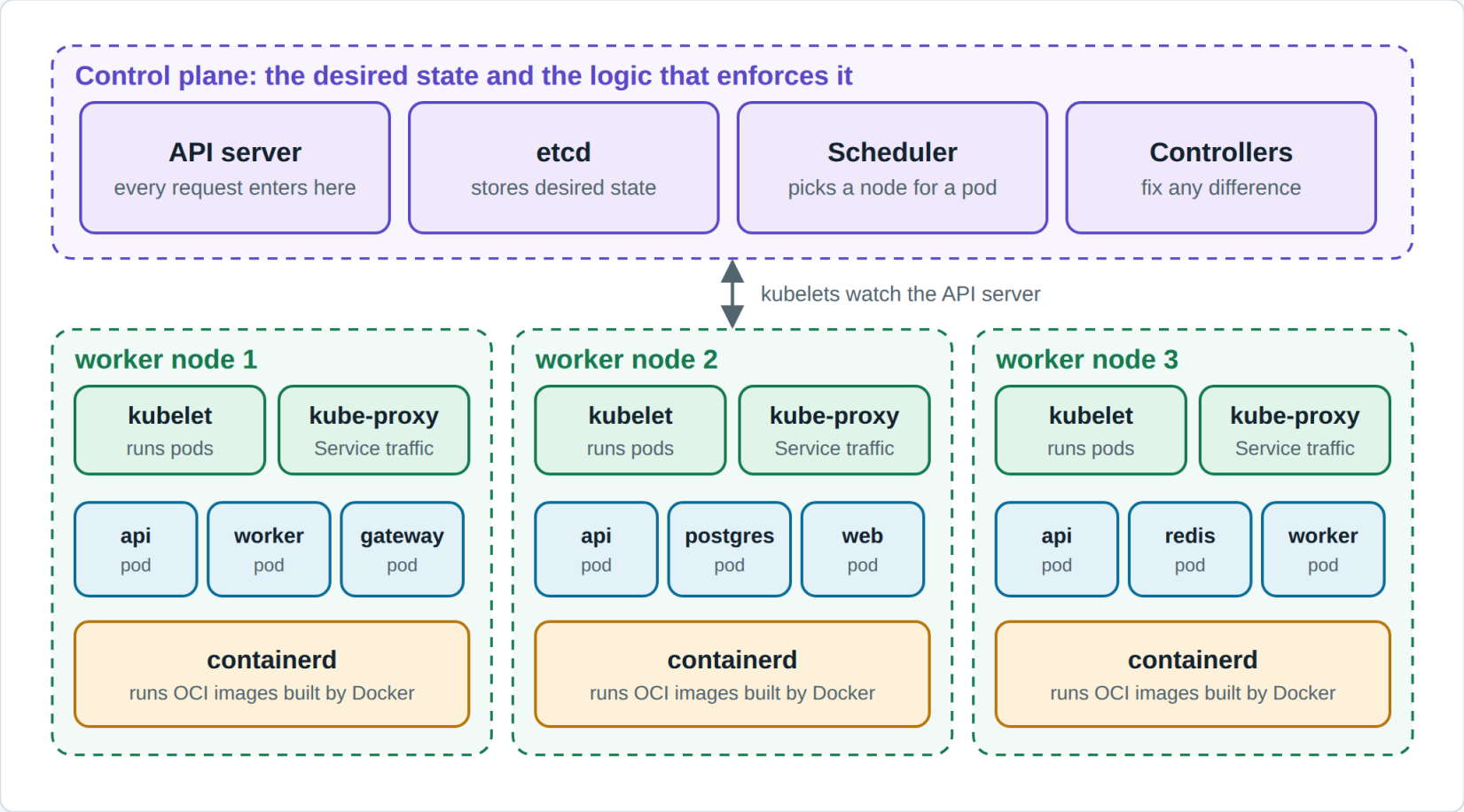
Restart policies

They restart a process that exited by itself. A container stopped with docker stop or docker kill stays stopped, by design.

Docker can report a container as unhealthy, but on one host it does not replace it. That is an orchestrator’s job.

THEORY

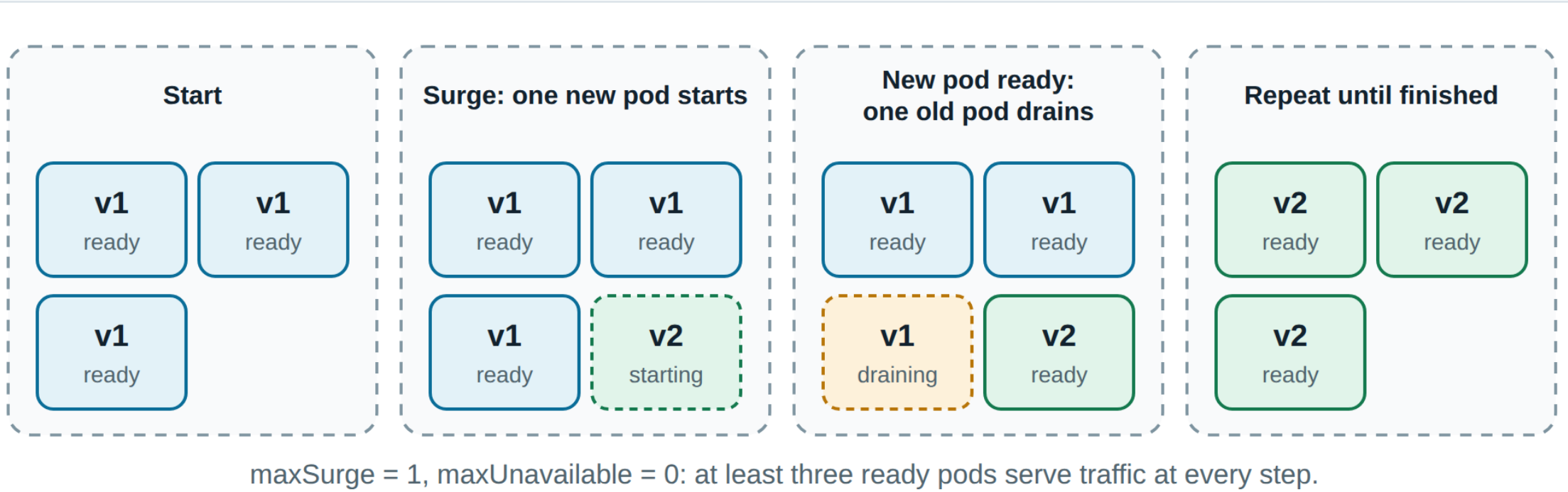
Kubernetes keeps reality equal to what you declared



- Declare the state you want
- Controllers fix any difference
- Rolling updates, probes, autoscaling

THEORY

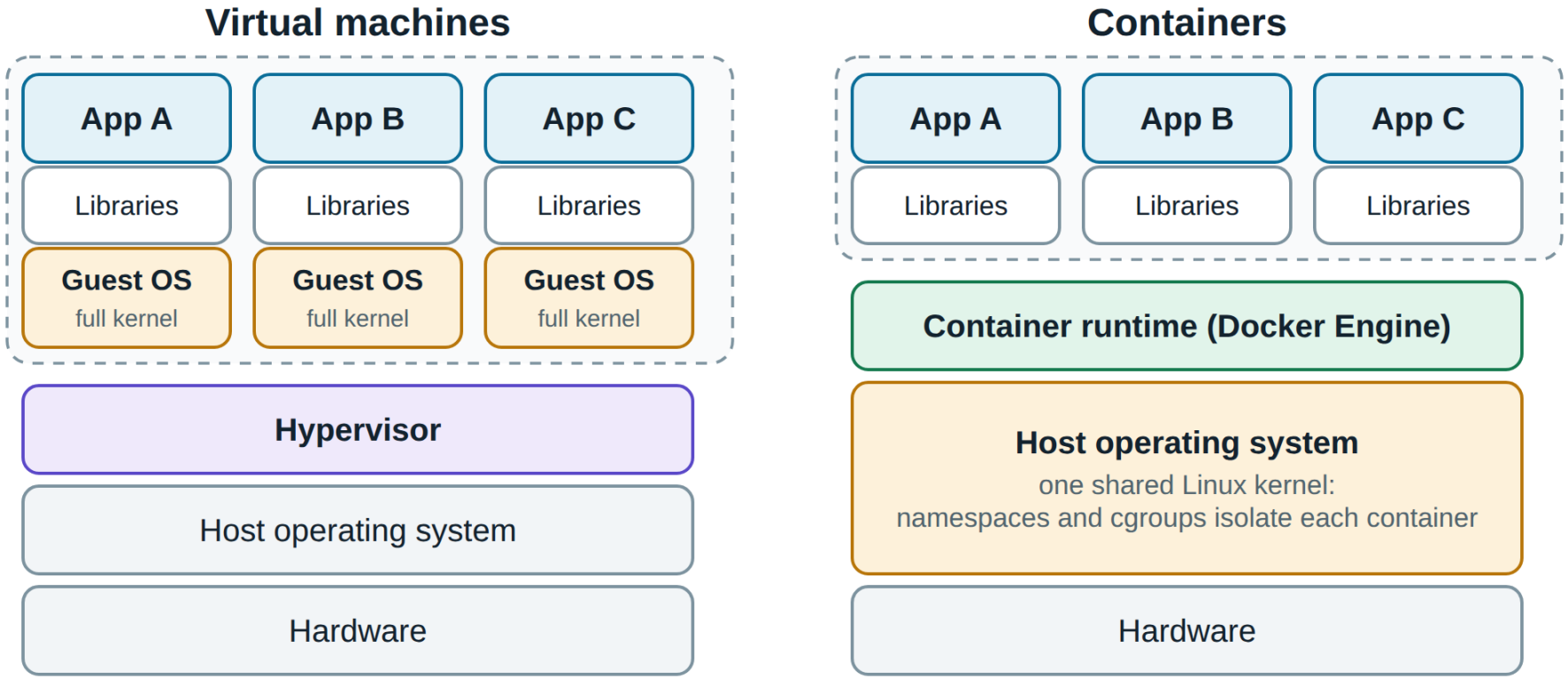
A rolling update replaces Pods one at a time



A new Pod must be ready before an old one leaves, and one command rolls the update back.

COMPARISON

Containers and virtual machines isolate at different levels



Containers are fast and small. VMs isolate more strongly. Clouds usually run containers inside VMs.

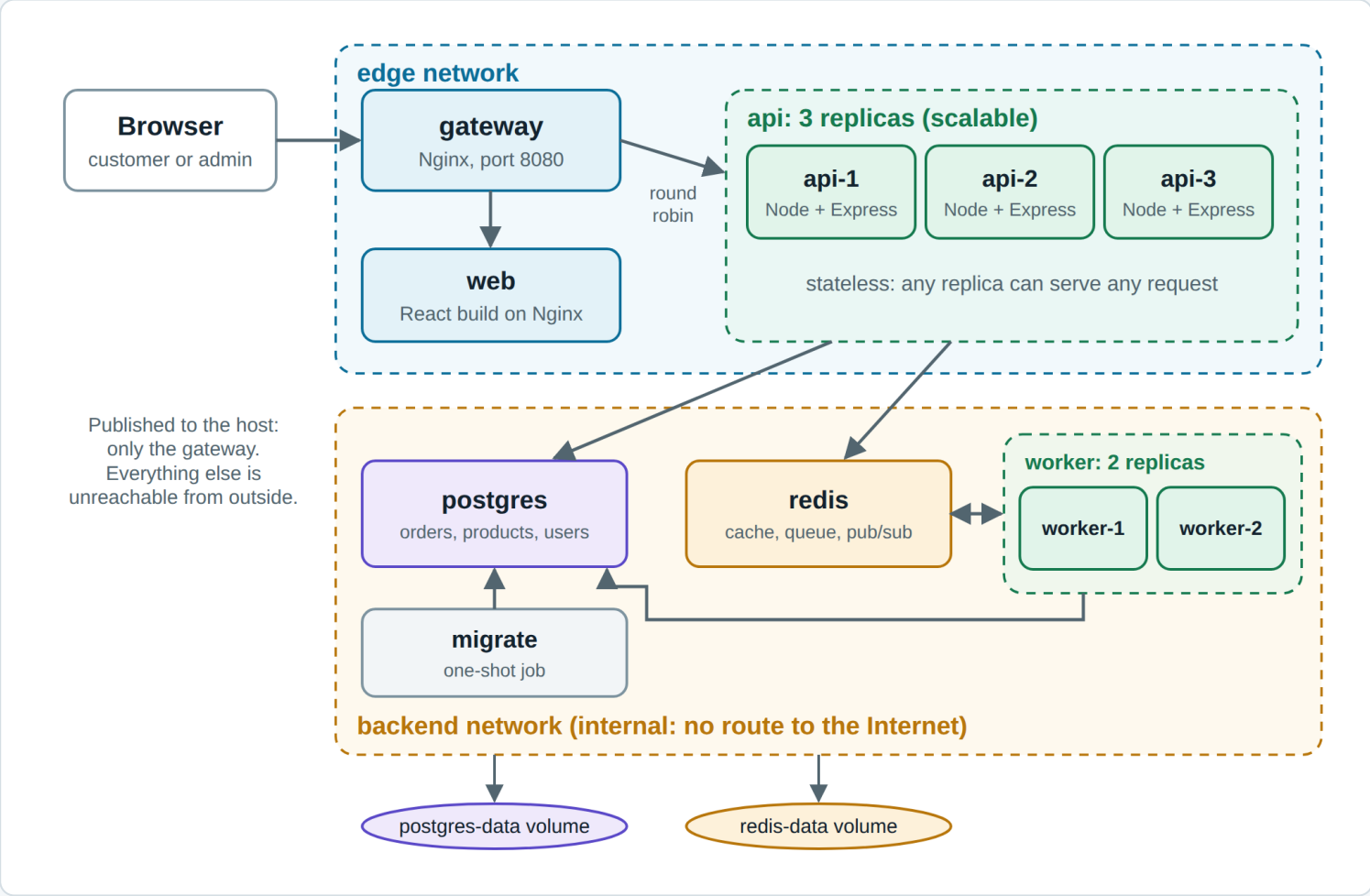
COMPARISON

Choose the orchestrator the problem needs

	Compose	Swarm	Kubernetes
Hosts	One	Many	Many
Learning curve	Low	Low	High
Self-healing	Restart on exit	Replaces tasks	Replaces and reschedules
Rolling update	None built in	Yes	Yes, with rollback
Autoscaling	Manual	Manual	Built in

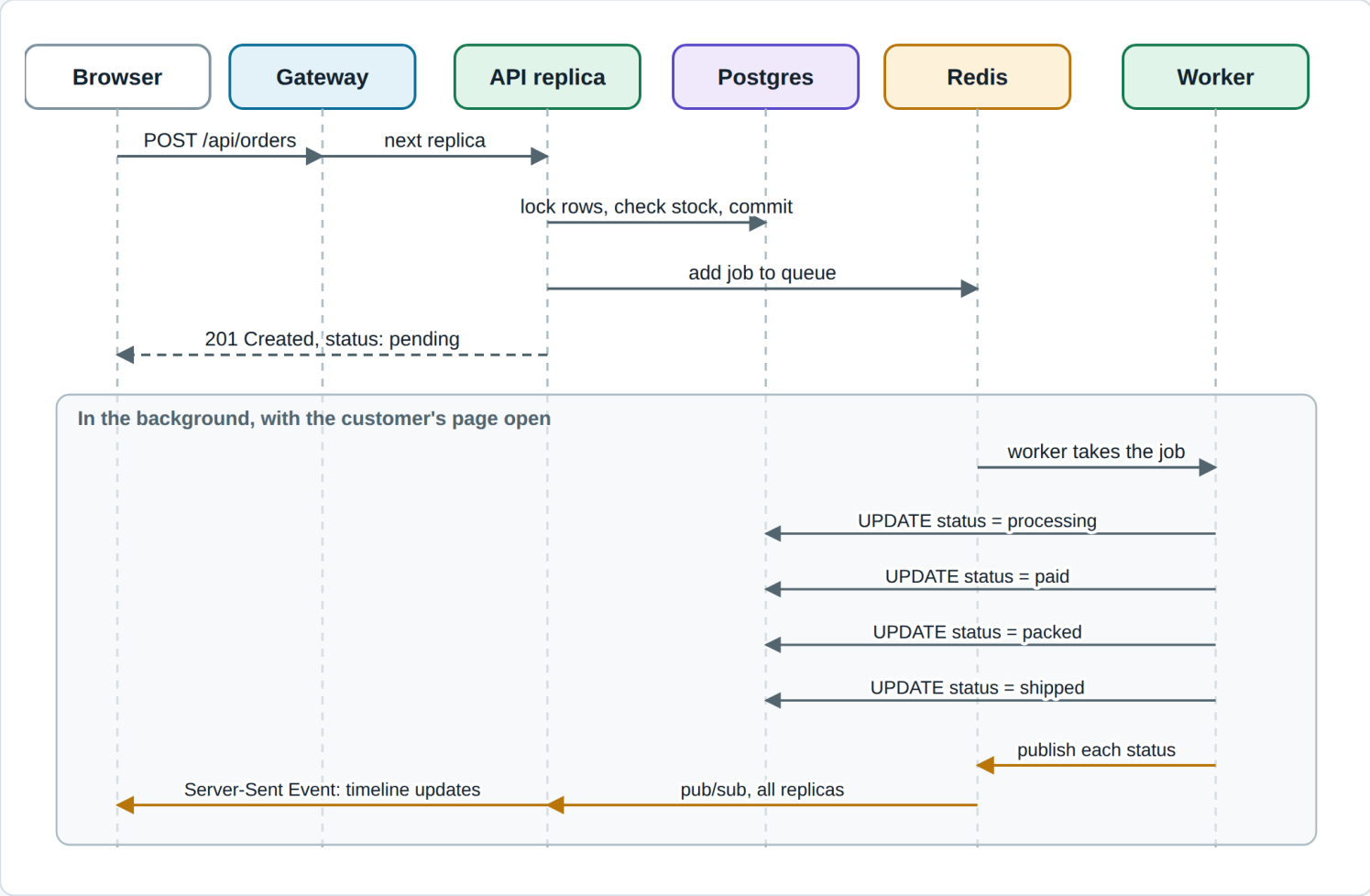
A small site is fine on Compose. Kubernetes pays off when you must survive losing a machine.

Seven kinds of container, one published port



- API is stateless, so it can scale
- One image, three commands
- Databases sit on an internal network

One order crosses every container



- Stock locked in a transaction
- Worker handles it in the background
- Status reaches the browser live

Statelessness is what makes scaling possible

State	Usual place	DockMart
User session	Server memory	Signed JWT in a cookie
Cached products	A variable	Redis, shared by replicas
Live connections	One process	Redis pub/sub
Background jobs	A timer	BullMQ queue
Business data	Local file	PostgreSQL volume

Hardening every container

Non-root

USER node in the image

Read-only

Only /tmp is writable

No capabilities

cap_drop: ALL

Secrets as files

Never in env or images

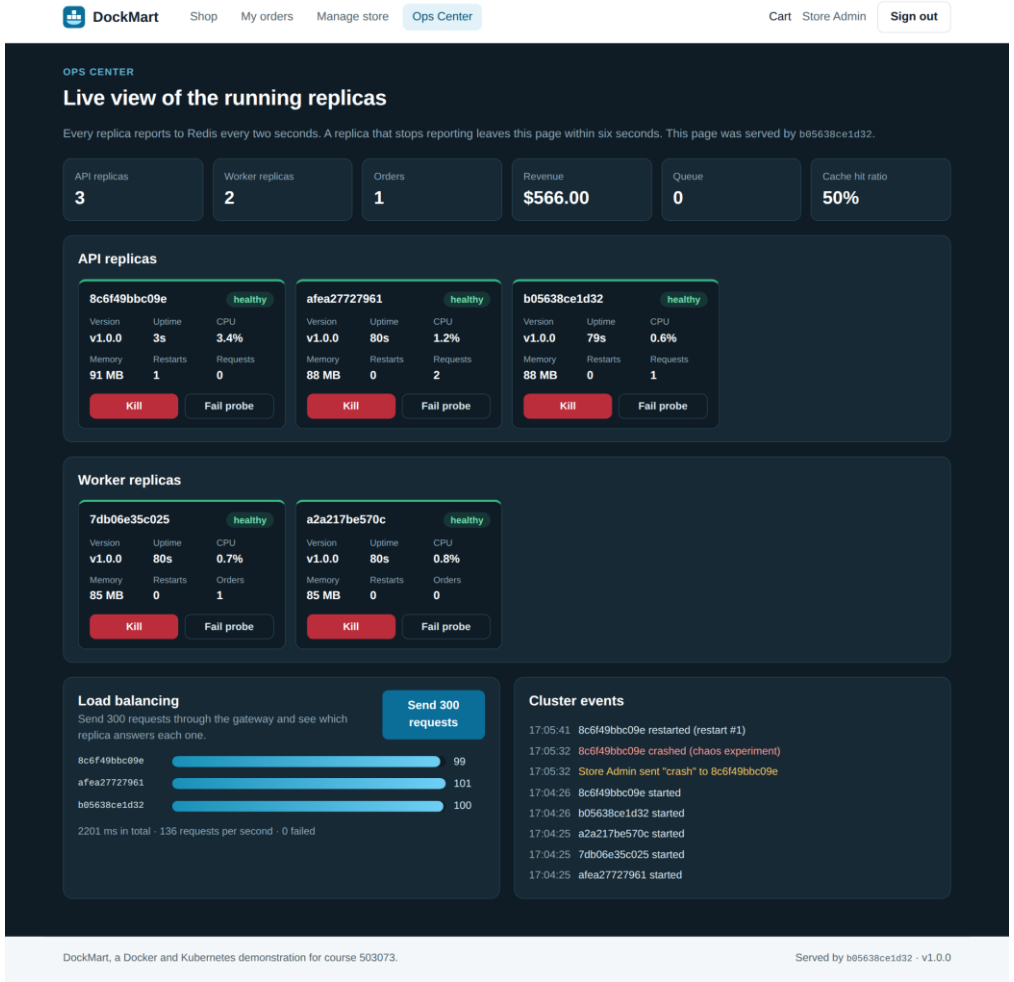
Isolated networks

Gateway cannot reach data

Resource limits

CPU and memory per service

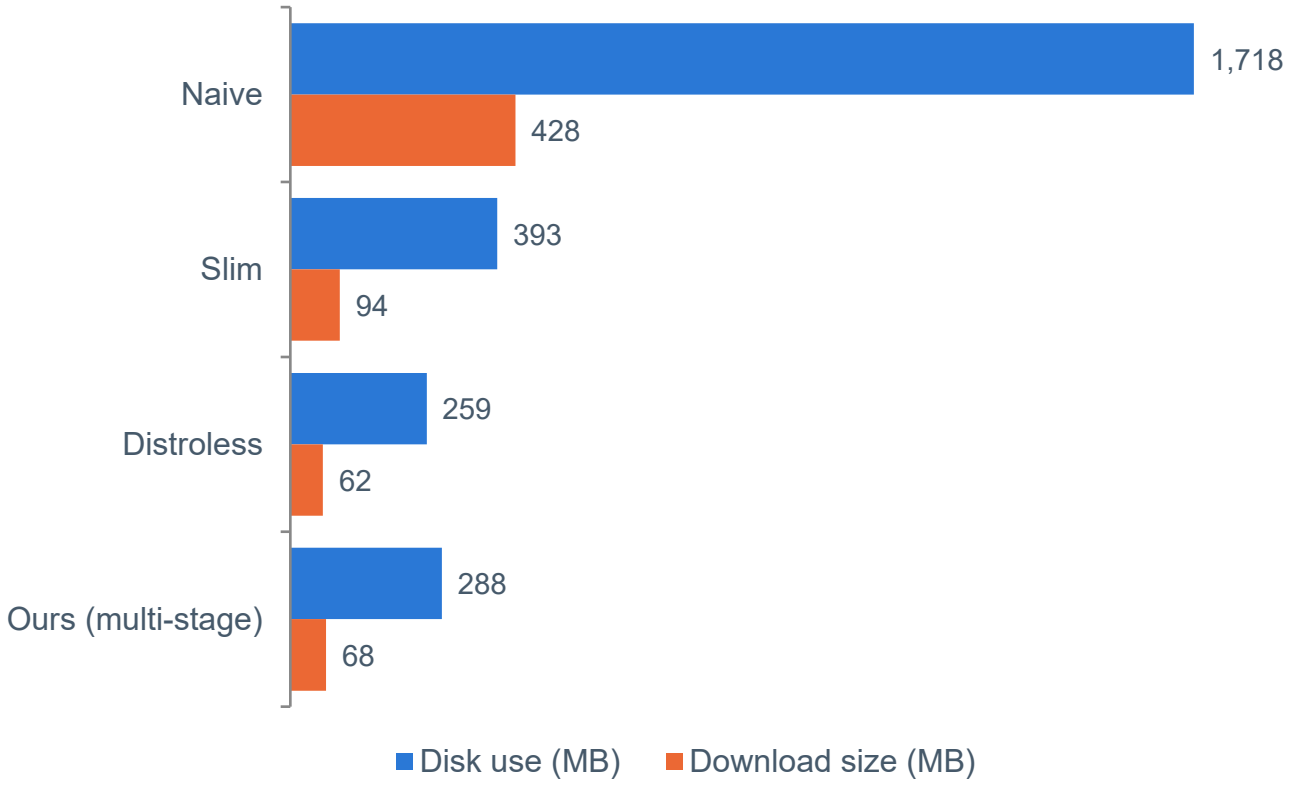
The Ops Center makes the cluster visible



- Every replica reports to Redis every 2 seconds
- Send 300 requests and watch them spread
- Kill a replica: uptime resets, restarts rises
- No Docker socket needed, so it also works on Kubernetes

RESULTS

The image got 6 times smaller and much cleaner

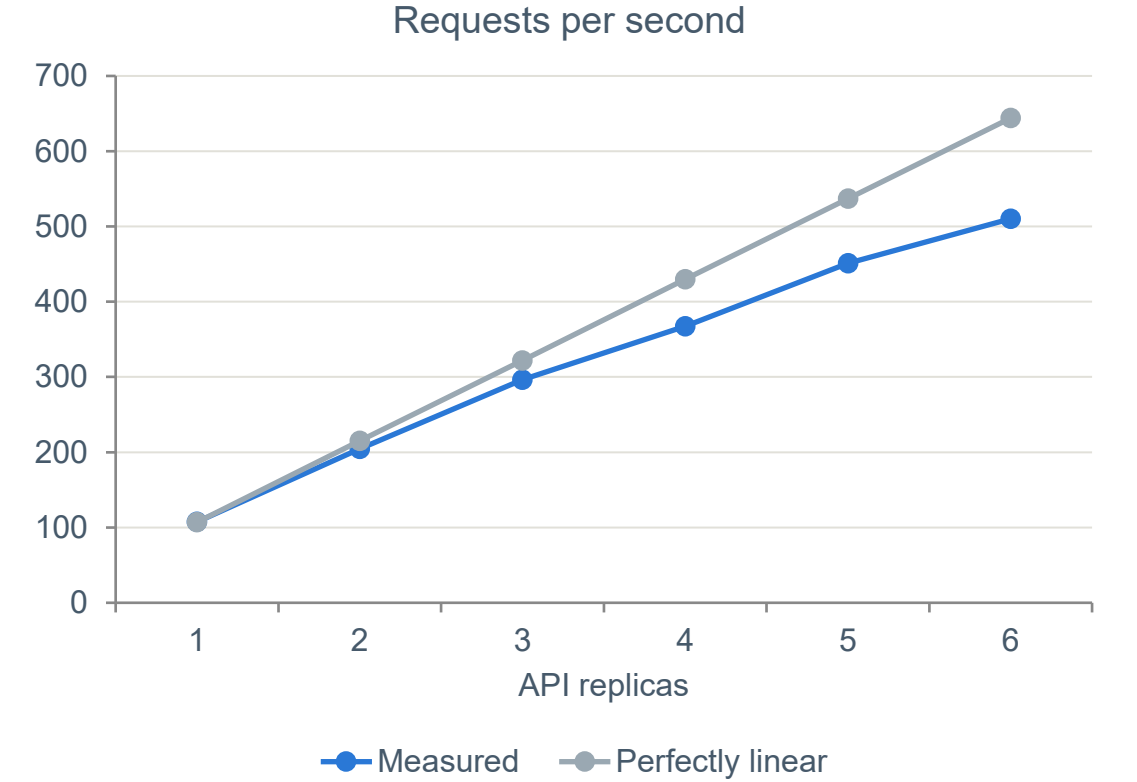


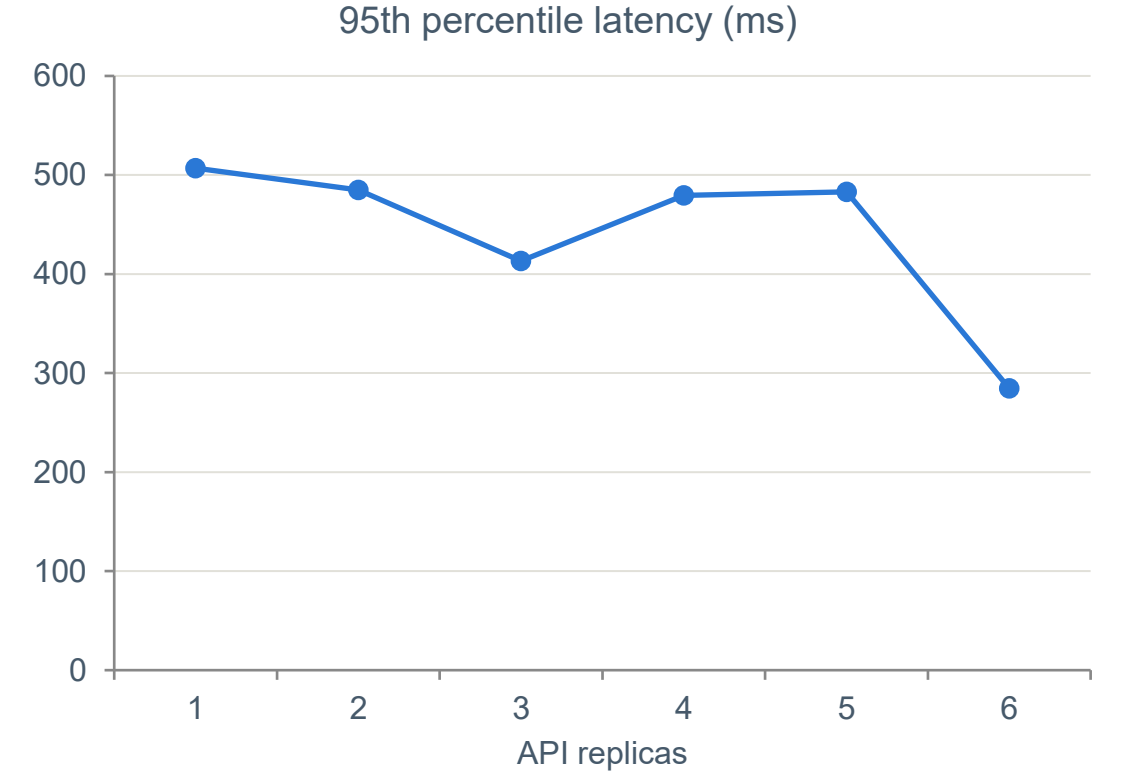
	Critical	High
Naive	57	373
Slim	2	76
Distroless	0	11
Ours	0	6

Vulnerabilities found by Grype. A package that is not in the image cannot be vulnerable.

RESULTS

Six replicas gave 4.8 times the throughput





No failed requests, and the gateway was never restarted. Each replica is limited to half a CPU core.

RESULTS

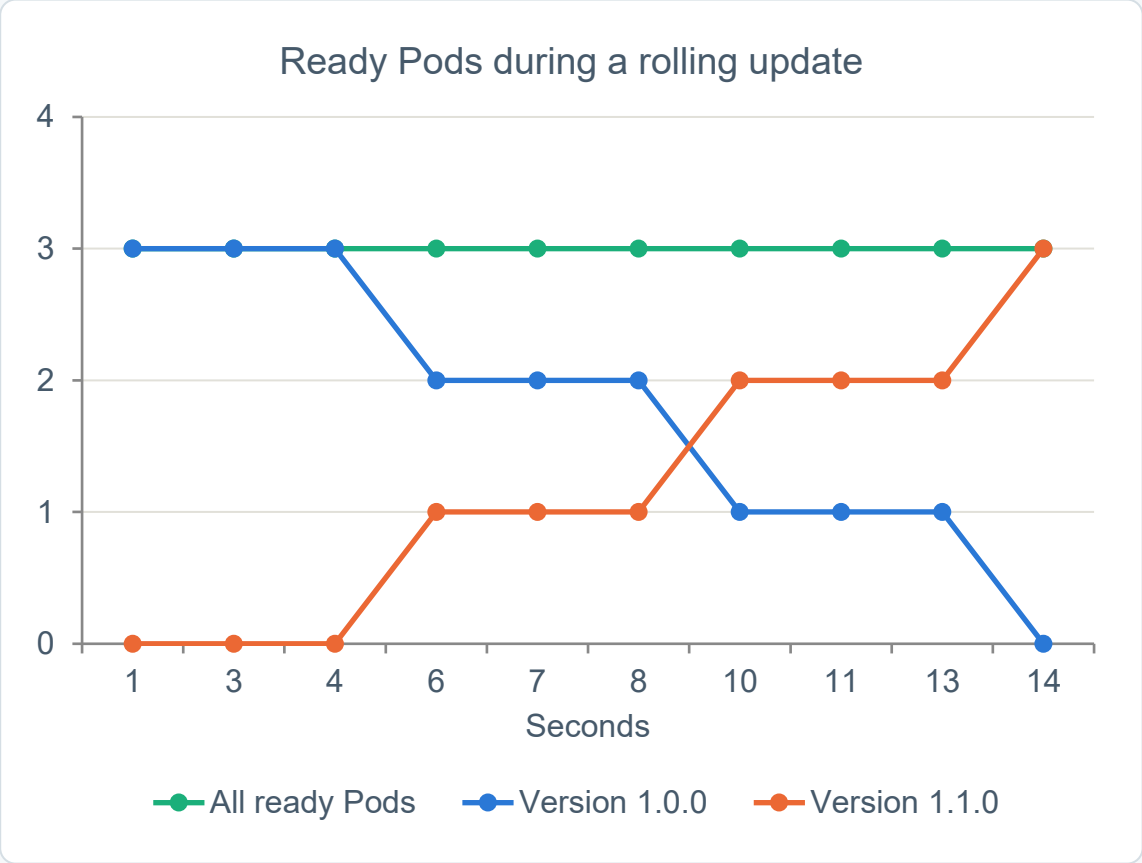
We broke it under load, and visitors noticed nothing

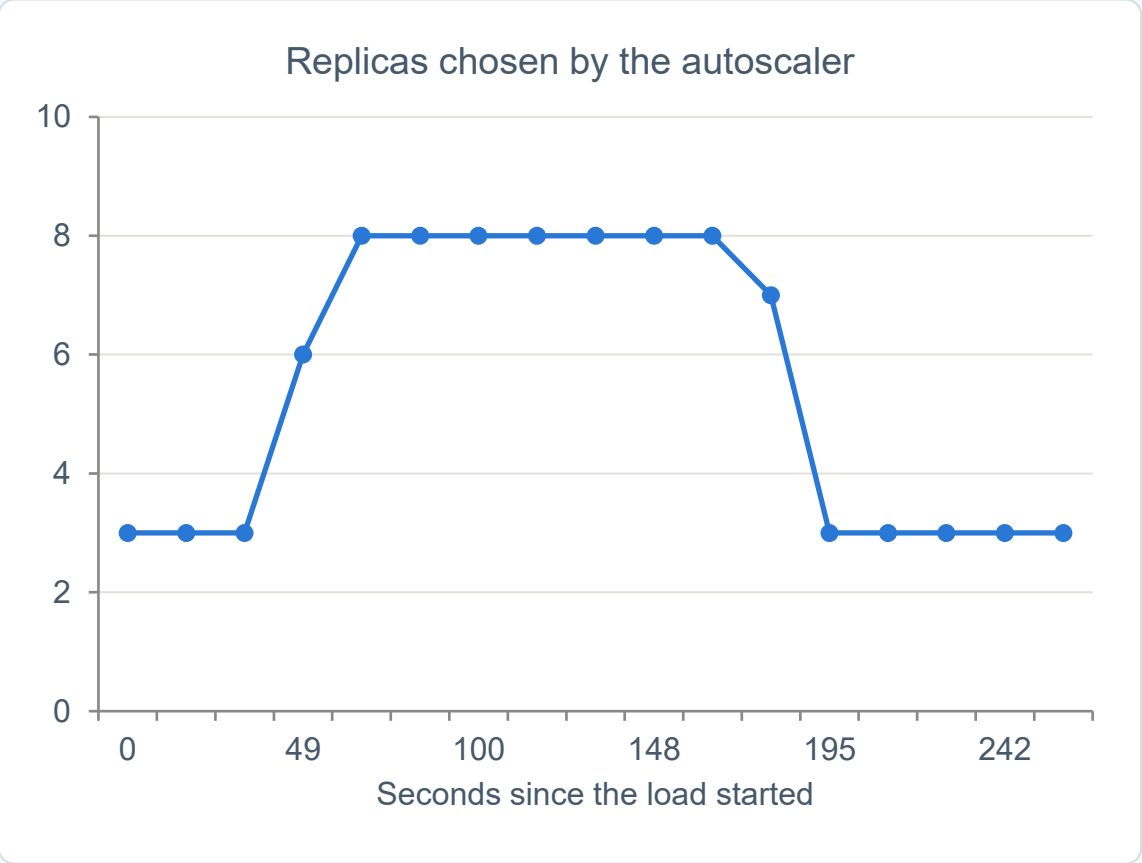
Failure, at 100 requests per second	Failed requests	Recovery
Process crash (Compose)	0 of 4,001	6.6 s
Rolling update (Kubernetes)	0 of 7,500	14.2 s
Rollback (Kubernetes)	0 of 7,501	15.2 s
Liveness failure (Kubernetes)	0 of 7,501	21.2 s
Node drained (Kubernetes)	0 of 7,501	16.7 s

During a Redis outage the catalog still answered in 28 ms. Only checkout was refused, with a clear message.

RESULTS

Kubernetes scaled itself and updated with zero downtime





Three ready Pods at every moment of the update. Autoscaling went from 3 to 8 replicas in about 60 s, and back with no intervention.

RESULTS

Testing failure found what happy paths hide

Redis outage froze the site

Fixed with fail-fast and a PostgreSQL fallback: 8.5 s down to 28 ms.

Load reached too few Pods

Nginx keeps connections open by default, and a Service balances per connection.

Liveness restart lost requests

Readiness now fails first, so traffic leaves the Pod early.

Worker could reach the Internet

Found by measuring every network path, then closed.

CONCLUSION

Containers deliver, if the application cooperates

Portability

The same images passed the same 24 tests on both platforms.

Efficiency

6 times smaller images, about 227 MB for nine containers.

Scalability

4.8 times the throughput, with zero-downtime updates.

Next: replicated databases, HTTPS at the edge, and a real cloud cluster.

Thank you

Questions are welcome.